Fake News & Misinformation Detection System

Final Report

Seminar Project Team

June 18, 2026

Contents

1 Business Problem Definition and Project Objective

1.1 Business Context and Motivation

Misinformation can spread faster than manual fact-checking teams can respond. In real operations such as media moderation, newsroom verification, and community trust-and-safety, the main pain points are high review volume, short response time, and inconsistent decision quality across reviewers. This project addresses that operational gap with an NLP-driven decision-support system that performs first-pass triage and provides evidence-aware explanations.

1.2 Target Users and Stakeholders

The intended users are moderators, verification staff, and analysts who need quick but reviewable assessments of article-like claims. Stakeholders include platform operators and project supervisors who evaluate whether the system is accurate, explainable, and deployable with limited resources.

1.3 Why NLP Is Required

The input is unstructured natural language text. Detecting misinformation requires understanding semantics, context, negation, and claim framing, which are not reliably captured by rule-only approaches. Therefore, NLP models are needed for scalable and consistent text understanding.

1.4 Success Metrics

The project uses measurable success criteria in both business and technical dimensions.

Table 1: Project success metrics

Metric Type	Metrics
Business metrics	Faster first-pass review, reduced manual triage effort, improved prioritization for human verification
Technical metrics	Accuracy, precision, recall, F1-score, API latency, uncertainty/conflict rate in agent workflow

2 Development Infrastructure and Software Engineering Practice

The project follows production-oriented software development rather than notebook-only experimentation. The implementation is in Python, version-controlled with Git, and organized as modular packages.

2.1 Repository Structure

The repository includes the expected production-style structure:

- `src/`: backend, agent workflow, model inference, data and monitoring modules.
- `data/`: data files and data scripts.
- `models/`: saved baseline and transformer artifacts.
- `configs/`: YAML-based configuration for model, data, training, monitoring, and retrieval.
- `tests/`: unit and API tests for workflow and endpoints.

2.2 Dependency and Reproducibility Setup

Dependencies are managed via `requirements.txt` and project metadata in `pyproject.toml`. The project includes Docker configuration for reproducible local deployment. The README documents environment setup, training, inference, and serving commands.

3 Data Management

3.1 Data Source and Licensing Notes

The pipeline is configured around a local FakeNewsNet-style CSV dataset with text and binary label columns. The project documents source assumptions and treats dataset licensing and provenance as part of responsible data handling.

3.2 Dataset Characteristics

The system is designed for English-language news-like text with labels `fake` and `real`. Dataset quality is evaluated through split consistency, class balance awareness, and post-training error inspection.

3.3 Preprocessing and Cleaning

Preprocessing includes HTML and URL removal, text normalization, duplicate filtering, and empty-text handling. These steps are implemented as reusable code modules to keep the pipeline deterministic and auditable.

3.4 Train/Validation/Test Strategy

Data is split into 70% train, 15% validation, and 15% test with stratification and fixed random seed. This supports fair comparison between baseline and transformer models.

3.5 Known Limitations and Bias Risks

Potential risks include topic imbalance, source-domain bias, and limited generalization to out-of-domain claims. These limitations are acknowledged in evaluation, monitoring, and ethics sections rather than hidden.

4 Model Selection and Optimization

4.1 Model Choice Justification

Two model families are used:

- Baseline: TF-IDF + Logistic Regression for interpretability and fast benchmarking.
- Main model: RoBERTa for contextual understanding and stronger semantic representation.

This combination satisfies both practical benchmarking and high-performance modeling requirements.

4.2 Training and Tuning Procedure

The baseline is trained with sparse features and class-balanced logistic regression. The transformer is fine-tuned from `roberta-base` with tuned learning rate, sequence length, and checkpoint selection based on validation F1. Hyperparameter tuning is performed at least at a basic level through configured search choices.

4.3 Baseline Comparison and Results

Table 2: Model evaluation summary

Model	Accuracy	Precision	Recall	F1
TF-IDF + Logistic Regression	0.9913	0.9938	0.9870	0.9904
RoBERTa	0.9991	0.9987	0.9997	0.9992

As shown in Table 2, RoBERTa outperforms the baseline across all reported metrics.

4.4 Error Analysis and Trade-offs

Observed failure modes include ambiguous claims, weak-evidence scenarios, and noisy out-of-domain phrasing. In the latest live retrieval hardening pass, the system was improved with claim-centric queries, domain-aware fallback, and direct NASA source recovery for science claims; however, external search providers can still miss the intended authoritative page or return noisy concept pages before the fallback path activates. The key trade-offs are:

- Accuracy vs speed: transformer is stronger but heavier than baseline.
- Complexity vs maintainability: agent-enabled pipeline is more useful but requires stronger integration testing and monitoring.
- Retrieval robustness vs dependency on external search providers: more aggressive fallback improves recall but increases integration complexity.

5 Deployment Architecture and Inference Pipeline

5.1 Deployment Format

The system provides both a REST API and a web-based demo:

- FastAPI backend with endpoints `/predict`, `/analyze`, `/agent`, `/health`, and `/version`.
- Streamlit frontend for user interaction and result visualization.

5.2 Input and Output Format

Input is article text. Output includes predicted label, confidence, evidence summary, trust score, and explanation fields (depending on endpoint).

5.3 Architecture Diagram

Figure 1 summarizes the deployed system.

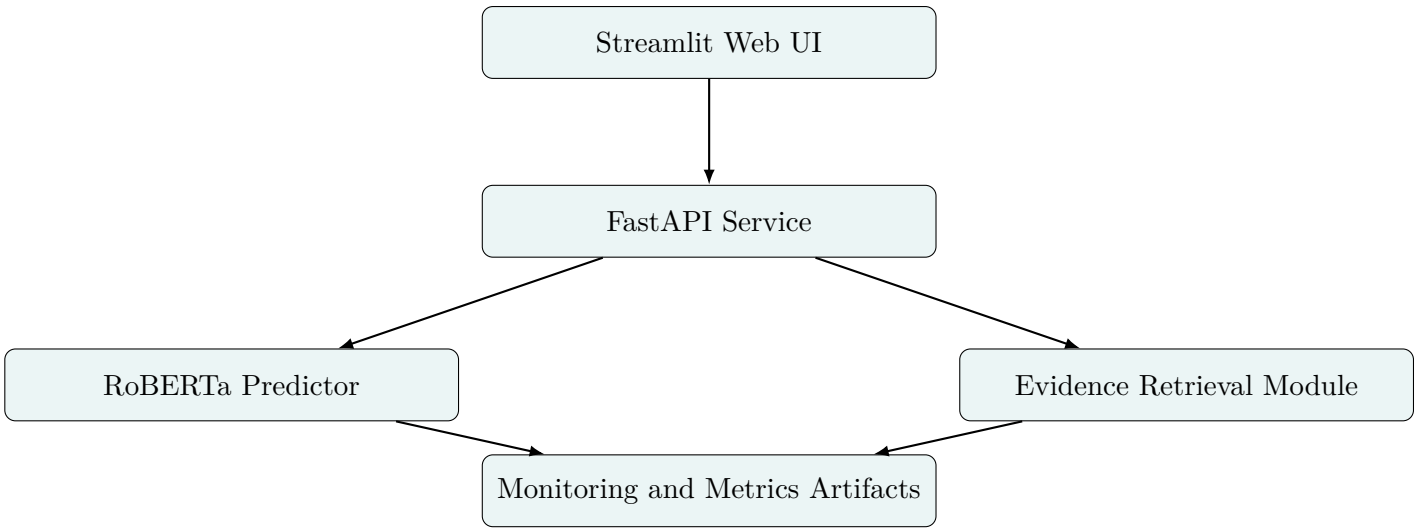

Figure 1: Deployed architecture of the fake news detection system.

5.4 Deployment Constraints and Limitations

Main constraints are free-tier resource limits, model size, and dependence on external evidence sources. These limitations are documented and reflected in risk management.

The live retrieval layer is intentionally conservative: when search quality is weak, the system may return `UNCERTAIN` rather than fabricate evidence. This is preferable for seminar evaluation, but it means some correct claims can still be misclassified or under-supported if the classifier is overconfident or the search provider is noisy.

6 Agentic AI Component

6.1 Agent Architecture

The project includes explicit multi-step agentic behavior through a controller that coordinates classifier, evidence retrieval, decision logic, and explanation generation.

6.2 Flow and Decision Logic

Figure 2 shows the implemented sequence.

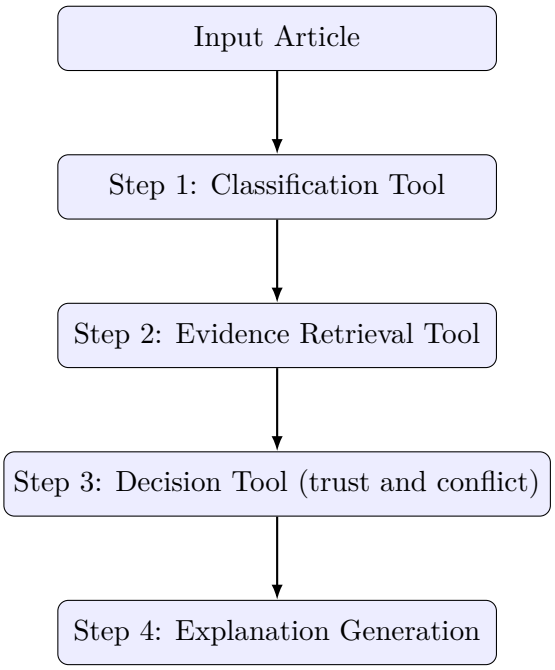

Figure 2: Agentic workflow with intermediate decision signals.

6.3 Example Interaction Behavior

The agent can emit an `UNCERTAIN` review state when confidence is low or evidence conflicts with prediction. This demonstrates decision-making based on intermediate outputs, not single-pass labeling. Recent runtime traces also show why this matters: a science claim may now recover

authoritative NASA evidence after fallback, while noisy claim wording can still initially attract irrelevant pages before the retrieval gate filters them out.

7 Continual Learning and Monitoring Strategy

The project includes a practical monitoring design and a conceptual continual-learning plan.

7.1 Monitoring Metrics

Table 3: Monitoring metrics and purpose

Metric	Purpose
Confidence trend	Detect confidence degradation over time
Prediction distribution drift	Identify label skew changes
Data drift indicators	Detect input pattern shifts
Agent uncertain/conflict rate	Track reliability of evidence-assisted decisions
Latency and error rate	Monitor deployment stability

7.2 Continual Learning Plan

New data would be collected from reviewed uncertain/error cases, validated by human checks, and added to periodic retraining batches. Retraining would be triggered by threshold breaches in drift and uncertainty metrics.

8 Data Privacy and Model Robustness

8.1 Privacy Handling

The system masks personally identifiable information before inference and logging, reducing exposure of sensitive strings in artifacts and logs.

8.2 Robustness Considerations

The system explicitly addresses noisy and out-of-domain inputs by:

- exposing uncertainty states,
- capturing evidence conflict,
- documenting failure cases for reviewer escalation.

8.3 Mitigation Strategy

Mitigations include conservative decision thresholds, human review escalation, and monitoring-driven retraining priorities.

9 Project Management and Teamwork Reflection

9.1 Project Plan and Task Breakdown

The project was executed in staged milestones: data and baseline setup, transformer training, agent integration, frontend/backend deployment, and final hardening.

Table 4: Team-style task breakdown (solo simulation)

Role Focus	Responsibilities
ML engineer role	data preprocessing, baseline and transformer training, evaluation artifacts
Backend engineer role	API design, inference integration, endpoint validation, monitoring hooks
Frontend engineer role	Streamlit interaction flow, result visualization, user-facing error handling
MLOps/release role	deployment scripts, Docker setup, reproducibility and final submission packaging

9.2 Scale-up Reflection

In a real team, this project would scale with clearer service ownership, CI/CD checks per component, and shared monitoring dashboards.

10 Ethics and Responsible AI

10.1 Who Benefits and Who May Be Harmed

Benefits include faster reviewer triage and more structured claim analysis. Potential harms include false positives impacting legitimate content and false negatives allowing misinformation to pass.

10.2 Bias and Fairness Risks

Bias can arise from dataset composition, source imbalance, and domain mismatch. The project mitigates this through explicit limitation reporting, error analysis, and cautious interpretation of model outputs.

10.3 Explainability for Non-technical Stakeholders

The UI presents prediction, trust, evidence summary, and uncertainty state in plain language to support human decision-making.

10.4 Potential Misuse

Possible misuse includes over-reliance on automated outputs without human verification. The project mitigates this by surfacing uncertainty and recommending human review for ambiguous cases.

11 Conclusion

This project satisfies the assignment requirements by delivering an end-to-end, production-minded NLP system with clear business framing, structured data management, baseline and transformer comparison, agentic reasoning, deployable interfaces, monitoring strategy, privacy handling, and ethical reflection.

The main technical achievement is not only high classification performance but also integration of evidence-aware decision logic and explainability to support trustworthy operation.

References

[1] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

[2] Liu, Y., Ott, M., Goyal, N., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.

[3] Shu, K., Mahudeswaran, D., and Liu, H. (2018). FakeNewsNet: A Data Repository with News Content, Social Context and Dynamic Information.

[4] FastAPI documentation. https://fastapi.tiangolo.com/

[5] Streamlit documentation. https://streamlit.io/